

Server TCP Multithread per la gestione dei movimenti bancari

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 14

Indice

1	Introduzione	2
2	Stato dell'arte	2
3	Metodologia usata	3
3.1	Struttura modulare del progetto	3
3.2	Struttura dati	3
3.3	Operazioni implementate	4
3.4	Gestione della concorrenza	4
3.5	Protocollo applicativo	4
3.6	Compilazione ed esecuzione	5
4	Risultati sperimentali	5
4.1	Test funzionali	5
4.2	Test di concorrenza	6
4.2.1	Multithread	7
4.2.2	Sincronizzazione e concorrenza	7
5	Conclusione e sviluppi futuri	8

1 Introduzione

L'obiettivo del progetto è realizzare un sistema client/server TCP multithread per la gestione dei movimenti bancari di un unico conto corrente. Ogni movimento è caratterizzato da un identificativo, un importo e una causale.

Il server deve permettere l'accesso concorrente da più punti della rete, come terminale bancario, applicazione mobile, sportello o servizio di e-banking.

Per questo motivo, è necessario gestire più client contemporaneamente, evitando che l'accesso simultaneo alla struttura dati condivisa produca inconsistenze.

Le operazioni supportate dal sistema sono:

- ADD, per inserire un nuovo movimento;
- DELETE, per eliminare un movimento esistente;
- UPDATE, per modificare un movimento;
- LIST, per visualizzare i movimenti registrati;
- QUIT, per uscire.

La soluzione proposta utilizza socket TCP, thread POSIX, un mutex per proteggere la memoria condivisa e un semaforo per limitare il numero massimo di client concorrenti a `MAX_NUM`.

2 Stato dell'arte

Il modello client/server è una delle architetture più utilizzate nelle applicazioni di rete. Un server mette a disposizione un servizio, mentre uno o più client si connettono per inviare richieste e ricevere risposte.

Nel progetto è stato utilizzato il protocollo TCP, che è orientato alla connessione e garantisce una comunicazione affidabile tra client e server.

La sequenza tipica lato server è:

- `socket()` -> creazione della socket;
- `bind()` -> associazione ad una porta;
- `listen()` -> messa in ascolto;
- `accept()` -> accettazione.

In un server sequenziale, ogni client verrebbe servito uno alla volta. Questa soluzione è semplice, ma poco adatta al caso considerato, in cui più accessi possono avvenire contemporaneamente.

Per questo motivo, è stato adottato un server multithread. Dopo ogni `accept()`, il server crea un nuovo thread dedicato alla gestione del client appena connesso. In questo modo, il server principale può continuare ad accettare nuove connessioni, mentre i thread worker elaborano i comandi dei client.

L'uso dei thread richiede però attenzione alla concorrenza. Se più thread accedono contemporaneamente alla stessa struttura dati, possono verificarsi race condition. Per evitarle, le sezioni critiche devono essere protette tramite meccanismi di sincronizzazione, come i mutex.

3 Metodologia usata

3.1 Struttura modulare del progetto

Il progetto è stato organizzato in più file, in modo da separare la logica bancaria, la comunicazione di rete e il protocollo applicativo.

Questa organizzazione rende il codice più leggibile e permette di modificare una parte del progetto senza influenzare direttamente le altre.

File	Ruolo
<code>common.h</code>	Contiene le costanti condivise del progetto, come la porta, la dimensione dei buffer e il valore <code>MAX_NUM</code> .
<code>bank.h</code> , <code>bank.c</code>	Definiscono e implementano la struttura dati bancaria e le operazioni sui movimenti.
<code>protocol.h</code> , <code>protocol.c</code>	Contengono le funzioni di supporto per inviare e ricevere messaggi tramite socket.
<code>server.c</code>	Implementa il server TCP multithread e la gestione dei client concorrenti.
<code>client.c</code>	Implementa il client interattivo da terminale.
<code>Makefile</code>	Automatizza la compilazione di server e client.

Tabella 1: Moduli principali del progetto

3.2 Struttura dati

I movimenti bancari sono memorizzati in un array statico. Ogni elemento contiene l'identificativo dell'operazione, l'importo e la causale.

La struttura dati condivisa è rappresentata da un database in memoria:

```
1  typedef struct {
2      Movement movements[MAX_MOVEMENTS];
3      int count;
4      pthread_mutex_t mutex;
5  } BankDB;
```

Codice 1: Struttura dati condivisa

Il campo `movements` contiene i movimenti registrati, `count` indica quanti movimenti sono attualmente presenti e `mutex` protegge l'accesso concorrente alla struttura.

L'uso di un array statico è stato usato perchè si adatta al contesto dell'esercitazione e permette di concentrarsi sui concetti principali: socket TCP, thread e sincronizzazione.

3.3 Operazioni implementate

Il server supporta le operazioni fondamentali richieste:

- `bank_add()`, che inserisce un nuovo movimento se l'identificativo non è già presente;
- `bank_delete()`, che elimina un movimento dato il suo identificativo;
- `bank_update()`, che modifica importo e causale di un movimento esistente;
- `bank_list()`, che restituisce la lista dei movimenti salvati.

Le operazioni lavorano tutte sulla stessa struttura dati condivisa. Per questo motivo l'accesso viene protetto tramite mutex, evitando modifiche simultanee da parte di più thread.

3.4 Gestione della concorrenza

Ogni client connesso viene gestito da un thread separato. Il thread riceve i comandi del client, li interpreta e invoca la funzione corrispondente sulla struttura dati bancaria.

La sezione critica è costituita dall'accesso all'array dei movimenti. Prima di leggere o modificare la struttura, il thread acquisisce il mutex; al termine dell'operazione lo rilascia.

```
1  pthread_mutex_lock(&db->mutex);
2
3  /* operazione sulla struttura dati condivisa */
4
5  pthread_mutex_unlock(&db->mutex);
```

Codice 2: Schema di protezione con mutex

Inoltre, il numero massimo di client gestiti contemporaneamente è limitato da un semaforo inizializzato a `MAX_NUM`. Prima di creare un nuovo thread, il server effettua una `sem_wait()`, mentre alla chiusura della connessione il thread effettua una `sem_post()`.

3.5 Protocollo applicativo

La comunicazione tra client e server avviene tramite un protocollo testuale. Il client invia un comando come stringa, il server lo interpreta e restituisce una risposta.

Le risposte del server indicano se l'operazione è andata a buon fine (OK *<tipo di operazione effettuata>*) oppure se si è verificato un errore(ERR *<tipo di errore>*).

Per le risposte multilinea, come LIST e HELP, viene utilizzata una riga finale END, affinché il client possa riconoscere la fine del messaggio.

3.6 Compilazione ed esecuzione

La compilazione del progetto è gestita tramite `Makefile`. I comandi principali sono:

```
make  
make clean
```

Codice 3: Compilazione del progetto

make compila,
make clean elimina gli eseguibili e i file oggetto.

4 Risultati sperimentali

Per verificare il corretto funzionamento del progetto sono stati eseguiti test di compilazione, test funzionali sulle operazioni principali e test con più client concorrenti.

4.1 Test funzionali

Sono state verificate le operazioni di inserimento, aggiornamento, eliminazione e visualizzazione dei movimenti.

Esempio di interazione client/server:

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho14/codice$ ./server  
Server TCP multithread avviato sulla porta 8080  
Numero massimo di client/worker contemporanei: 10  
Client connesso: 127.0.0.1:59216  
Client disconnesso.
```

```

alessia23@aleb23:~/LabReSid24-25/hands-on/ho14/codice$ ./client
Connesso al server della banca.

Comandi disponibili: (ricorda di scriverli in maiuscolo)
  ADD <id> <importo> <causale>
  DELETE <id>
  UPDATE <id> <nuovo_importo> <nuova_causale>
  LIST
  HELP
  QUIT

> LIST
Nessun movimento registrato.
> ADD 1 150 regalo
OK Movimento aggiunto.
> ADD 2 300 stipendio
OK Movimento aggiunto.
> ADD 3 30.40 bonifico
OK Movimento aggiunto.
> UPDATE 2 400.30 aggiunta stipendio
OK Movimento aggiornato.
> LIST
Movimenti presenti: 3
ID = 1 | Importo = 150.00 | Causale = regalo
ID = 2 | Importo = 400.30 | Causale = aggiunta stipendio
ID = 3 | Importo = 30.40 | Causale = bonifico
> DELETE 3
OK Movimento eliminato.
> LIST
Movimenti presenti: 2
ID = 1 | Importo = 150.00 | Causale = regalo
ID = 2 | Importo = 400.30 | Causale = aggiunta stipendio
> QUIT
OK Connessione chiusa.

```

I test mostrano che le operazioni principali vengono elaborate correttamente dal server.

4.2 Test di concorrenza

Per verificare il comportamento multithread, e la concorrenza sono stati fatti due test:

4.2.1 Multithread

Sono stati avviati 10 client

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho14/codice$ ./server
Server TCP multithread avviato sulla porta 8080
Numero massimo di client/worker contemporanei: 10
Client 1 connesso: 127.0.0.1:33802
Client 2 connesso: 127.0.0.1:33810
Client 3 connesso: 127.0.0.1:44214
Client 4 connesso: 127.0.0.1:44218
Client 5 connesso: 127.0.0.1:44224
Client 6 connesso: 127.0.0.1:44236
Client 7 connesso: 127.0.0.1:44250
Client 8 connesso: 127.0.0.1:57180
Client 9 connesso: 127.0.0.1:57190
Client 10 connesso: 127.0.0.1:57198
Client 10 disconnesso.
Client 11 connesso: 127.0.0.1:57214
```

L'undicesimo client ha ricevuto questo messaggio:

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho14/codice$ ./client
WAIT Server pieno: attendi che si liberi uno slot...
```

Appena un client si disconnette e quindi si libera uno slot l'11 client riesce a connettersi:

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho14/codice$ ./client
WAIT Server pieno: attendi che si liberi uno slot...
Connesso al server della banca.

Comandi disponibili: (ricorda di scriverli in maiuscolo)
  ADD <id> <importo> <causale>
  DELETE <id>
  UPDATE <id> <nuovo_importo> <nuova_causale>
  LIST
  HELP
  QUIT

> █
```

4.2.2 Sincronizzazione e concorrenza

Sono stati avviati due client su macchine diverse, uno Linux e uno macOS, collegati allo stesso server.

Entrambi hanno inviato quasi contemporaneamente un comando **ADD** con lo stesso identificativo.

Il server ha accettato solo una dei due movimenti, mentre l'altro è stato rifiutato perché l'identificativo era già presente.


```

alessia23@aleb23:~/LabReSid24-25/hands-on/ho14/codice$ ./client
Connesso al server della banca.

Comandi disponibili: (ricorda di scriverli in maiuscolo)
  ADD <id> <importo> <causale>
  DELETE <id>
  UPDATE <id> <nuovo_importo> <nuova_causale>
  LIST
  HELP
  QUIT

> ADD 1 120 stipendio
OK Movimento aggiunto.
> LIST
Movimenti presenti: 1
ID = 1 | Importo = 120.00 | Causale = stipendio

```

```

alessiabrunello@Mac codice % ./client 192.168.1.34 8080
Connesso al server della banca.

Comandi disponibili: (ricorda di scriverli in maiuscolo)
  ADD <id> <importo> <causale>
  DELETE <id>
  UPDATE <id> <nuovo_importo> <nuova_causale>
  LIST
  HELP
  QUIT

> ADD 1 200 regalo
ERR Esiste già un movimento con ID 1.
> LIST
Movimenti presenti: 1
ID = 1 | Importo = 120.00 | Causale = stipendio

```

Quindi il server ha gestito correttamente le connessioni multiple. La struttura dati condivisa è rimasta coerente grazie all'uso del mutex, che impedisce modifiche simultanee non controllate.

L'ordine di visualizzazione dei movimenti potrebbe variare, poiché i thread vengono schedulati dal sistema operativo in modo non deterministico.

5 Conclusione e sviluppi futuri

Il progetto ha permesso di realizzare un server TCP multithread per la gestione di movimenti bancari. Il sistema consente a più client di collegarsi contemporaneamente ed eseguire operazioni di aggiunta, eliminazione, aggiornamento e visualizzazione.

L'uso del mutex consente di proteggere la struttura dati condivisa, mentre il semaforo limita il numero massimo di client concorrenti.

In conclusione, il programma soddisfa gli obiettivi dell'esercitazione e rappresenta una buona applicazione dei concetti di socket TCP, thread e sincronizzazione.

Tra i possibili sviluppi futuri si possono considerare:

- salvataggio persistente dei movimenti su file o database;
- gestione di più utenti o più conti correnti;
- autenticazione dei client;
- uso di una struttura dati dinamica al posto dell'array statico;
- logging delle operazioni eseguite;